



UNIVERSIDAD DE CONCEPCIÓN
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA CIVIL INDUSTRIAL



SISTEMAS INTELIGENTES DE PRODUCCIÓN ANALISIS: A HYBRID GENETIC TABU SEARCH ALGORITHM FOR SOLVING JOB SHOP SCHEDULING PROBLEMS: A CASE STUDY

5.0

Grupo 4

Javiera Montesinos Abarca
Maximiliano Pérez Luarte
Alan Willson Germany

Introducción a Sistema Inteligentes
de Producción
2025-2

01

INTRODUCCIÓN AL PROBLEMA TEÓRICO

I

El problema abordado es el **Job Shop Scheduling Problem (JSSP)**

- problemas más estudiados
- Mas desarrollado en programación determinística

II

El JSSP es de **naturaleza combinatoria** y **NP-hard**.

III

El objetivo de optimización es determinar los tiempos de inicio (t_{ik}) y fin de las operaciones para minimizar el makespan (C_{max}).

Restricciones clave del JSSP:

1. **Precedencia**: seguir un orden dado.
2. **Capacidad**: procesar solo una operación a la vez.
3. **No Preempción**: No son posibles las interrupciones

Aunque la literatura abarca variantes como el FJSP, el enfoque de este proyecto se centra en el JSSP clásico y determinístico.

02

INTRODUCCIÓN AL PROBLEMA PRÁCTICO

Instancias originales del estudio (paper)

JSSP1: Un problema de 8 trabajos por 6 máquinas (8 x 6).

JSSP2: Un problema de 6 trabajos por 6 máquinas (6 x 6).

JSSP3: Un problema de 6 trabajos por 6 máquinas (6 x 6).

Obtención y Representación

La instancia utilizada en este proyecto se basa en casos de estudio de la vida real provenientes del sector de molinos de acero (steel mill), donde se producen piezas de recambio.

El problema práctico surge porque no existen sistemas de programación estandarizados.

Como resultado, los cronogramas presentaban largos tiempos de inactividad en las máquinas, extendiendo innecesariamente la finalización de los trabajos.

02 INTRODUCCIÓN AL PROBLEMA PRÁCTICO

Instancias generadas para análisis de sensibilidad

JSSP1 (+20%):

Una variante del problema de 8 trabajos por 6 máquinas donde se incrementan los tiempos de procesamiento en 20% para simular mayor carga. (8×6).

JSSP2 (+20%):

La misma estructura del taller, pero reduciendo los tiempos en 20% para representar un escenario de menor saturación. (8×6).

Propósito de estas instancias

Estas dos variantes permiten evaluar la sensibilidad del sistema frente a cambios en la carga del taller, comparando la estabilidad del MAS y el rendimiento del modelo MIP ante perturbaciones controladas.

MODELO MATEMÁTICO DEL PROBLEMA (MIP)

El modelo matemático utilizado como referencia óptima (baseline) fue un modelo exacto centralizado de MIP. Este modelo fue desarrollado previamente y utilizado para obtener los valores óptimos de C_{max} (517, 444 y 379)

El modelo MIP incluye:

VARIABLES DE TIEMPO

Tiempos de inicio ($S_{i,k}$),
tiempos de finalización ($C_{i,k}$)
y Makespan (C_{max})



VARIABLES BINARIAS DE PRECEDENCIA:

$y_{(i,k),(p,q)}$, utilizadas para imponer el orden en las máquinas.

MODELO MATEMÁTICO DEL PROBLEMA (MIP)

La formulación disyuntiva clásica define el problema de la siguiente manera:

CONJUNTOS Y PARÁMETROS

$J = \{1, \dots, n\}$: trabajos

$M = \{1, \dots, m\}$: máquinas

$O_i = \{1, \dots, m_i\}$: operaciones del trabajo i

$\tau_{i,k}$: duración de la operación O_{ik}

$M_{i,k}$: máquina asignada a la operación O_{ik}

M : constante grande para restricciones disyuntivas

VARIABLES

$S_{i,k}$: inicio de O_{ik}

$C_{i,k}$: término de O_{ik}

C_{max} : makespan

$y_{(i,k),(p,q)} \in \{0, 1\}$: orden entre operaciones en la misma máquina

FUNCIÓN OBJETIVO

Min C_{max} = Minimizar Makespan

RESTRICCIONES

(1) PRECEDENCIA

$$S_{i,k+1} \geq C_{i,k}$$

Garantiza que las operaciones de un mismo trabajo se ejecuten en el orden correcto.

(2) CAPACIDAD DE MÁQUINA

$$S_{i,k} \geq C_{p,q} - M(1 - y_{(i,k),(p,q)})$$

$$S_{p,q} \geq C_{i,k} - My_{(i,k),(p,q)}$$

Impide que dos operaciones que requieren la misma máquina se ejecuten simultáneamente.

(3) DEFINICIÓN DE TIEMPOS

$$C_{i,k} = S_{i,k} + \tau_{i,k}$$

Con esto se garantiza que la fecha de término refleja exactamente la duración real del proceso.

(4) MAKESPAN

$$C_{max} \geq C_{i,k}$$

Determina que el makespan sea al menos el tiempo de término de la operación más tardía.

(5) NO NEGATIVIDAD

$$S_{i,k} \geq 0$$

Evita tiempos negativos.

DESARROLLO

PSEUDOCÓDIGO IMPLEMENTADO

Algoritmo MAS-SMART (Instance)

INICIO

Entrada:

- Conjunto de trabajos J
- Para cada trabajo $j \in J$: secuencia ordenada de operaciones O_j
- Para cada operación: máquina requerida m y tiempo de proceso p

Inicialización:

Para cada trabajo j :

Crear un **JobAgent** con:

- Índice de operación actual $k = 1$
- Tiempo disponible ($ready_time$) = 0
- Tiempo total restante = suma de sus procesamientos

Para cada máquina m :

Crear un **MachineAgent** con:

- Tiempo disponible ($available_from$) = 0
- Bandeja de solicitudes ($inbox$) vacía
- Política SMART con parámetros α , β , γ

$total_ops \leftarrow$ número total de operaciones en la instancia

$ops_programadas \leftarrow 0$

04

DESARROLLO

Salida:

- Cronograma final para cada máquina
- Makespan = max(finish sobre todas las operaciones)

FIN

PSEUDOCÓDIGO IMPLEMENTADO

Mientras ops_programadas < total_ops hacer:

1. Los trabajos solicitan servicio a máquinas

Para cada JobAgent j que no haya finalizado:

$op_actual \leftarrow$ operación actual de j

Si op_actual existe entonces:

- máquina objetivo $\leftarrow$ máquina requerida por op_actual
- Enviar solicitud al MachineAgent correspondiente (incluyendo ready_time y remaining_proc)

2. Cada máquina decide qué operación ejecutar

progreso $\leftarrow$ FALSO

Para cada MachineAgent M :

$(op, start, finish) \leftarrow M.decidir()$

Si M selecciona alguna operación entonces:

- Actualizar ready_time y remaining_proc del JobAgent correspondiente
- Registrar $(j, k, start, finish)$ en el cronograma de M
- $ops_programadas \leftarrow ops_programadas + 1$
- progreso $\leftarrow$ VERDADERO

3. Prevención de estancamiento

Si progreso = FALSO entonces:

- Si todos los trabajos finalizaron $\rightarrow$ terminar algoritmo
- Sino $\rightarrow$ continuar a siguiente iteración

04 DESARROLLO

DEFINICIÓN DE AGENTES

JobAgents (Trabajos)

- Representan cada trabajo del sistema.
- Atributos: secuencia de operaciones, índice actual, ready_time, remaining_proc.
- Rol: solicitar procesamiento a la máquina requerida y actualizar su progreso.
- Interacción: envían solicitudes con su estado y reciben confirmación de ejecución.

MachineAgents (Máquinas)

- Representan cada recurso del sistema.
- Atributos: available_from, bandeja de solicitudes (inbox), allowed_ops, agenda local (schedule).
- Rol: resolver conflictos entre solicitudes aplicando la política SMART.
- Interacción: reciben solicitudes, calculan puntajes, seleccionan operación y notifican al JobAgent.

INTERACCIÓN Y COORDINACIÓN

- Todos los trabajos envían solicitud → todas las máquinas aplican SMART → actualizan agendas.
- No existe un coordinador central
- Mensajes mínimos:
 - a. solicitud (Job→Machine)
 - b. confirmación (Machine→Job).

POLÍTICA IMPLEMENTADA

SMART (EST, SPT y remaining work)

$$score = \alpha \cdot EST + \beta \cdot p + \gamma \cdot Rem$$

05

RESULTADOS

- Se evaluaron **3 instancias base** del paper y **2 instancias escaladas** ($\pm 20\%$).
- MIP se ejecutó 1 vez por instancia; MAS en 5 corridas (todas deterministas: mismo Cmax).
- El MIP reprodujo exactamente los valores óptimos del paper.

01

JSSP1 (8×6):

- MIP = 505 (óptimo), MAS = 522 $\rightarrow$ brecha +3%.
- Menor congestión $\rightarrow$ reglas locales se comportan mejor.

02

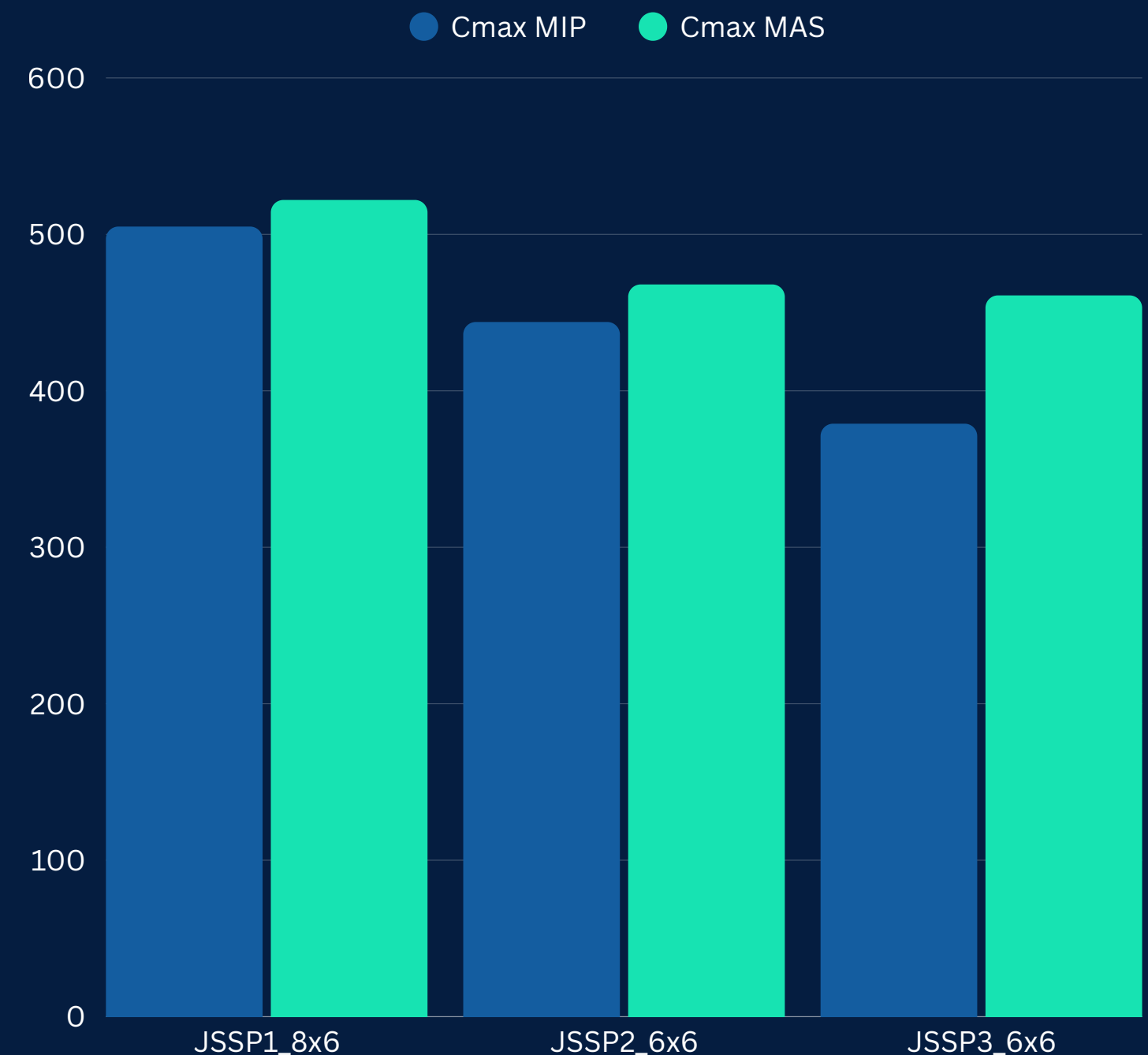
JSSP2 (6×6):

- MIP = 444, MAS = 468 $\rightarrow$ brecha +5%.
- Más colisiones en máquinas críticas $\rightarrow$ SMART pierde eficiencia.

03

JSSP3 (6×6):

- MIP = 379, MAS = 461 $\rightarrow$ brecha +22%.
- Alta densidad de precedencias $\rightarrow$ SMART genera colas en cuellos de botella.

**Tiempos de cómputo:**

- MIP: hasta 180 [s].
- MAS: 0.0002 [s] promedio .

05

RESULTADOS

JSSP1_FAST (-20%)

- **Cmax MIP:** 404
- **Cmax MAS:** 416
- **Brecha MAS vs MIP:** +2.97%
- **Tiempo MIP:** 180 s (llegó al límite)
- **Tiempo MAS:** 0.0000 s

JSSP2_SLOW (+20%)

- **Cmax MIP:** 532
- **Cmax MAS:** 561
- **Brecha MAS vs MIP:** +5.45%
- **Tiempo MIP:** 2.38 s
- **Tiempo MAS:** 0.0000 s

COMPORTAMIENTO OBSERVADO

- En la instancia rápida, el MAS se acerca más al MIP (menor congestión).
- En la instancia lenta, la brecha aumenta por mayor carga y más colas.
- El MAS mantuvo tiempos despreciables y resultados deterministas en todas las corridas.
- El MIP mostró **mayor sensibilidad** cuando aumentó la carga.



CONCLUSIONES

- ▶ El MIP obtiene soluciones óptimas, pero con tiempos de cómputo muy altos.
- ▶ El MAS es extremadamente rápido y consistente, pero su calidad depende de la política local.
- ▶ SMART funciona bien en instancias de baja y media congestión, pero pierde desempeño en estructuras más complejas.
- ▶ Ambos métodos muestran comportamientos complementarios: exactitud vs velocidad.

TRABAJO FUTURO

- **Ajustar la política SMART:** mejorar los pesos o permitir que se adapten según la instancia (evitar colas en máquinas críticas).
 - **Incorporar aprendizaje:** usar Q-learning o aprendizaje por refuerzo para que las máquinas aprendan mejores decisiones en entornos congestionados.
-
- **Mejorar coordinación global:** agregar señales ligeras entre máquinas para evitar decisiones locales que produzcan cuellos de botella globales.
 - **Probar instancias más grandes o dinámicas:** evaluar si la ventaja de velocidad del MAS se mantiene en problemas de escala industrial
-
- **Construir híbrido entre MAS + MIP:** usar MIP para dar una solución inicial o límites, y MAS para ajustar en tiempo real.